教學實作：擴充 SG90 伺服馬達功能

本章節實作一個「具備物理空間感」的伺服馬達控制功能。

步驟 1：修改硬體定義提示詞 (Devices Definition)

讓 AI 認識伺服馬達及其物理規則。

```
- SG90 Servo Motor: Pin 2 (Control range: 0 - 180 degrees).
- Physical Rules: Increasing values correspond to "clockwise" rotation, while decreasing values correspond to "counter-clockwise" rotation.
```

步驟 2：定義工具 JSON 規範 (Tools Definition)

在 tools_definition 中加入 /servo 指令的格式規範。

```
// If the system determines that the user expects assistance in controlling a servo motor, it must return only this valid JSON object:
{
  "type": "tool_call",
  "method": "/servo",
  "params": {
    "pin": "<Device pin number. If the user explicitly specifies a pin number, use that number. If the user does not specify which device pin to control, first ask the user in normal conversational reply and wait for the user's answer before filling this value. If multiple pins are mentioned, choose only the first one in order of appearance.>",
    "angle": "<Desired absolute angle from 0 to 180>"
  }
}
```

步驟 3：底層 C++ 程式碼實作

在專案中新增 tool_servo 驅動函數。

```
#include <AmebaServo.h>
AmebaServo myServo;

// Control a servo motor's position by specifying a target angle.
// This function supports precise physical movement for actuators like the SG90 servo connected to GPIO pins.
String tool_servo(int pin, int angle) {
    angle = constrain(angle, 0, 180);

   myServo.attach(pin);
    myServo.write(angle);

    return "Servo on pin " + String(pin) + " moved to " + String(angle) + " degrees.";
}
```

步驟 4：更新工具分發器邏輯 (useTools)

將 /servo 方法註冊進系統的分發流程。

```
void useTools(String command, JsonObject params) {
    if (command == "/on" || command == "/off" || command == "/pwm") {
        // ...
    }
    else if (command == "/servo") {
        int pin = params["pin"].as<int>();
        int angle = params["angle"].as<int>();

        // Execute hardware action
        String response = tool_servo(pin, angle);

        // Update conversation history (Memory Persistence)
        historical_messages += buildHistoricalData("user", command);
        historical_messages += buildHistoricalData("model", response);

        // Save to SD card (Persistence)
        storeHistoricalMessagesToFile();

        // Generate natural language response for the user
        response = Gemini_chat_request("espond only with a natural language description of the device's current operating state in the user's language. Never output tool_call, JSON, or any structured format. If there are any unfinished or pending tasks, clearly inform the user and ask whether to continue processing them.", 0);
        sendMessageToTelegram(telegramBot_token, telegramBot_chatID, response, "");
    }
}
```